OBJECT ORIENTED PROGRAMMING
CCS20704, DCS20504, CCS23803

Received Date : **3 October 2026**
Submission Due Date: **19 November 2026**
Lecturer : **Dr. Jamal Abdullahi**
Weightage : **15%**
Semester : **SEPTEMBER 2026**

Instruction to students:

- This is a GROUP assignment.
- Complete this cover sheet and attach it to your assignment (first page).

Student declaration:		
I declare that: • ***This assignment is my/our own work.*** • ***I/we understand what is meant by plagiarism.*** • ***My lecturer has the right to deduct my marks in case of:*** - ***Late submission*** - ***Any plagiarism found in my assignment.***		
		
NAME: THAVAANITHAN A/L SUNDRALINGAM **MATRICS NUMBER: 012026021321**	**NAME: VHETHAASHINI A/P THANGARAJU** **MATRICS NUMBER: 012025092401**	

1.0 INTRODUCTION

X-DRIVE is a Smart Car Rental and Management System developed using Java. The purpose of this project is to provide a simple and organised platform for managing vehicle information, customer details, rental transactions, payments, and vehicle returns.

Traditional car rental management may involve manually recording customer information, checking vehicle availability, calculating rental charges, and maintaining transaction records. These processes can become difficult to manage when the number of vehicles and customers increases.

To address these challenges, X-DRIVE provides a graphical user interface (GUI) that allows users to perform the main car rental operations through a single application. The system is designed using Object-Oriented Programming (OOP) principles, including encapsulation, inheritance, polymorphism, abstraction, and object creation.

The application uses Java Swing to develop its graphical interface. The system also applies a class-based structure to separate vehicle management, customer information, rental transactions, and payment processing.

Through this project, the group demonstrates how OOP concepts can be applied to develop a practical car rental management application.

2.0 PROJECT OBJECTIVES

The main objectives of the X-DRIVE system are:

1. To develop a Java-based car rental management application using Object-Oriented Programming concepts.
2. To create a base class and derived classes to represent different types of vehicles.
3. To demonstrate inheritance by allowing vehicle subclasses to reuse common attributes and methods from the base class.
4. To demonstrate polymorphism through a common interface and different implementations.
5. To apply encapsulation by protecting class attributes and controlling access through getter and setter methods.
6. To implement constructors for creating and initialising objects.
7. To develop additional classes to manage customers, rentals, and payments.
8. To apply conditional statements and loops in the system's operations.
9. To provide a graphical user interface that makes the system easier to use.
10. To improve the organisation and management of vehicle rental activities.

3.0 SYSTEM OVERVIEW

X-DRIVE is a desktop-based application developed using Java. It allows users to manage the main activities involved in a car rental business.

The application contains several main sections:

- **Dashboard:** Displays an overview of the rental system, including vehicle and rental information.
- **Vehicle Management:** Allows users to view and add vehicle information.
- **Customer Management:** Allows users to register and manage customer details.
- **Rental and Payment:** Allows users to select an available vehicle, create a rental transaction, calculate charges, and record payment information.
- **Returns and History:** Allows users to manage vehicle returns and review rental transaction records.

The system is designed with a graphical interface to make the operations more convenient. Users can interact with the application through buttons, tables, text fields, selection boxes, and other Swing components.

3.1 Development Tools and Technologies

Component	Description
Programming Language	Java
Development Environment	Visual Studio Code
Java Version	JDK 19.0.1
GUI Framework	Java Swing
Application Type	Desktop Application
Main Class	CarRentalApp
GUI Class	CarRentalGUI

3.2 System Scope

The scope of X-DRIVE focuses on the basic operations of a car rental management system. It includes vehicle registration, customer management, rental processing, payment recording, and vehicle returns.

The application is developed as an academic demonstration of Object-Oriented Programming. Payment methods displayed in the application represent payment options within the system and do not constitute direct integration with actual banking or e-wallet services.

4.0 SYSTEM DESIGN AND CLASS STRUCTURE

The X-DRIVE application uses multiple classes to organise the system according to their responsibilities. Each class represents a specific part of the car rental process.

4.1 Base Class: Car

The Car class acts as the main base class for vehicle objects. It contains common vehicle information and methods that can be shared by different vehicle types.

The class represents the general properties of a vehicle in the rental system.

Examples of attributes include:

- Vehicle ID
- Brand
- Model
- Year
- Daily rental rate
- Mileage
- Fuel type
- Availability status

The Car class provides constructors, getter methods, setter methods, and other vehicle-related operations.

4.2 Derived Class: Sedan

The Sedan class extends the Car class. It inherits the common vehicle attributes and methods while providing additional information or behaviour specific to sedan vehicles.

This allows the system to represent sedan vehicles without repeating all the attributes and methods already defined in the Car class.

4.3 Derived Class: SUV

The SUV class also extends the Car class. It represents sport utility vehicles and inherits the common vehicle properties from the base class.

The SUV class can contain its own additional attributes or methods to distinguish it from other vehicle types.

4.4 Additional Classes

The system includes additional classes to support the main rental operations.

Customer Class

The Customer class represents a customer who uses the car rental service. It stores customer-related information and provides methods to access or update the information.

Rental Class

The Rental class represents a rental transaction. It connects the customer and selected vehicle with the rental period and transaction details. It also supports the management of rental status and vehicle returns.

Payment Class

The Payment class represents payment information associated with a rental transaction. It supports the recording and management of payment details.

PaymentMethod

The PaymentMethod component represents the payment method options used by the application. It helps organise the payment choices available during the rental process.

4.5 Main Class: CarRentalApp

The CarRentalApp class acts as the main application class. It manages the main collections of vehicles, customers, rentals, and payments.

It also coordinates the operations between the different classes and starts the graphical user interface.

4.6 Graphical User Interface: CarRentalGUI

The CarRentalGUI class is responsible for displaying the application interface using Java Swing.

It provides the dashboard, navigation menu, tables, forms, buttons, and other graphical components. It also connects user actions to the relevant application operations.

5.0 IMPLEMENTATION OF OBJECT-ORIENTED PROGRAMMING CONCEPTS

X-DRIVE demonstrates the main Object-Oriented Programming concepts through its class structure and application operations.

5.1 Encapsulation

Encapsulation is the process of combining data and methods within a class while restricting direct access to the internal data.

In X-DRIVE, class attributes are protected using access modifiers such as private. Other classes access or modify these attributes through public getter and setter methods.

For example, vehicle information is managed through the Car class instead of allowing other classes to directly change its internal attributes.

This approach improves data protection, reduces unwanted modifications, and makes the program easier to maintain.

5.2 Inheritance

Inheritance allows one class to acquire the attributes and methods of another class.

In X-DRIVE, the Car class acts as the parent class, while Sedan and SUV act as derived classes.

The Sedan and SUV classes extend the Car class and reuse its common vehicle information and operations.

This reduces code duplication and makes it easier to introduce additional vehicle categories in the future.

5.3 Polymorphism

Polymorphism allows different objects to be treated through a common type while providing different implementations of the same method.

In X-DRIVE, the Rentable interface provides a common contract for rental-related behaviour. Different vehicle classes can implement the required method according to their own requirements.

This allows the application to work with different vehicle types through a common interface instead of requiring completely separate operations for every vehicle category.

5.4 Abstraction

Abstraction focuses on representing essential behaviour while hiding unnecessary implementation details.

In X-DRIVE, the Rentable interface defines the common rental behaviour that implementing classes are expected to provide.

The application can use this common contract without depending on every internal detail of each vehicle type.

5.5 Constructors

Constructors are used to initialise objects when they are created.

The Car class contains a constructor that initialises the vehicle information. Other classes also use constructors to initialise their respective objects.

For example, when a vehicle is registered, its object is created with the required vehicle details.

Constructors help ensure that objects are created with the information required by the application.

5.6 Getter and Setter Methods

Getter methods retrieve private attribute values, while setter methods allow controlled updates to those values.

In X-DRIVE, getter and setter methods are used to manage information such as vehicle details and other object attributes.

This supports encapsulation and allows validation to be applied when information is changed.

5.7 Object Creation and Interaction

The system creates objects from its classes to perform actual operations.

For example:

- A Car-derived object represents a vehicle.
- A Customer object represents a registered customer.
- A Rental object represents a rental transaction.
- A Payment object represents a payment record.

The main application manages these objects through collections and method calls. This demonstrates how different objects can work together to complete a real-world process.

6.0 SYSTEM FEATURES AND FUNCTIONALITIES

6.1 Dashboard

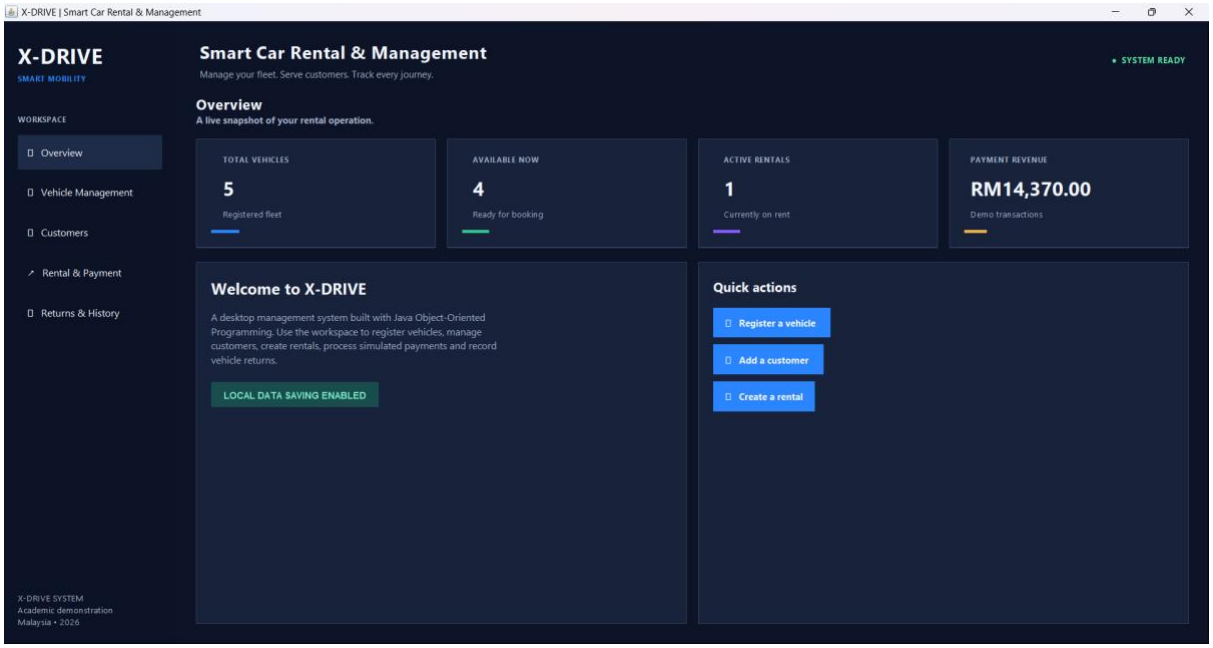


Figure 6.1.1: Main dashboard and navigation.

The dashboard provides an overview of the application. It displays important information about the rental system and provides navigation to the main sections.

6.2 Vehicle Management

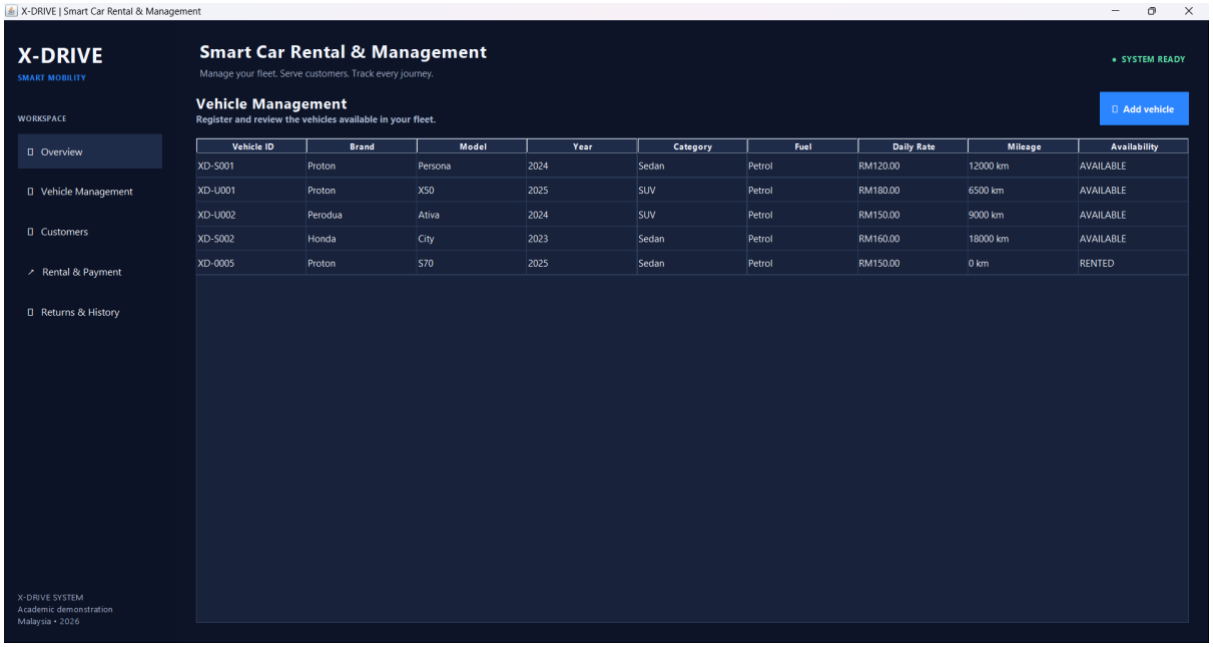


Figure 6.2.1: Vehicle Management

The vehicle management section allows users to view vehicle records and add new vehicles.

Vehicle information includes the vehicle ID, brand, model, year, rental rate, and availability status.

The system supports different vehicle categories, including sedans and SUVs.

6.3 Customer Management

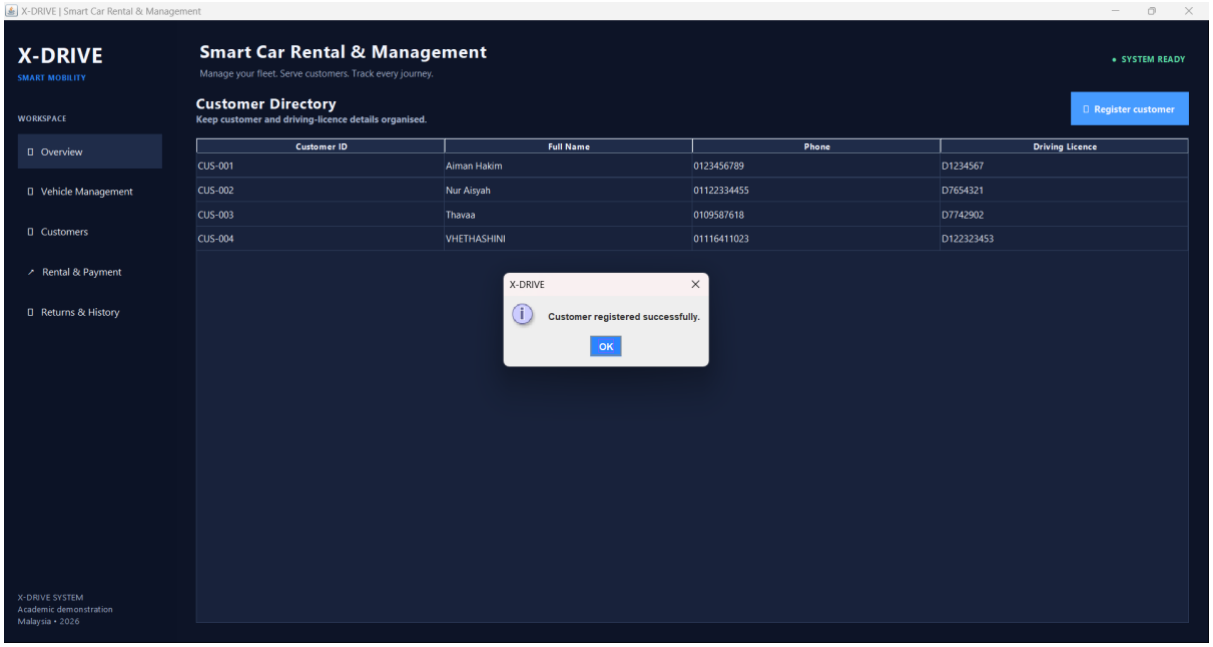


Figure 6.3.1: Customer Management

The customer management section allows users to register customers and view customer information.

This helps organise customer records and makes it easier to associate customers with rental transactions.

6.4 Rental and Payment

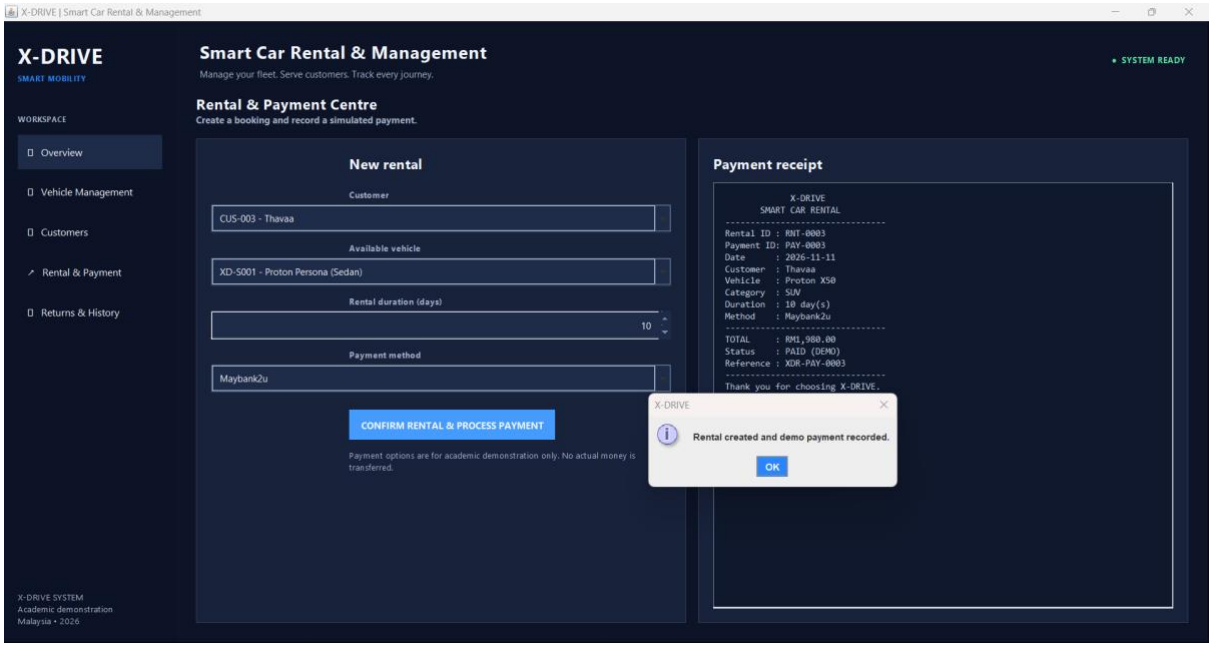


Figure 6.4.1: Rental and Payment

The rental and payment section allows users to select a customer and an available vehicle before creating a rental transaction.

The application calculates the rental amount based on the daily rental rate and selected rental duration. It also provides payment options for recording the payment method.

For example, a vehicle with a daily rental rate of RM180 rented for ten days produces a basic rental amount of RM1800. With the 10% surcharge applied in the tested transaction, the total becomes RM1980.

6.5 Returns and History

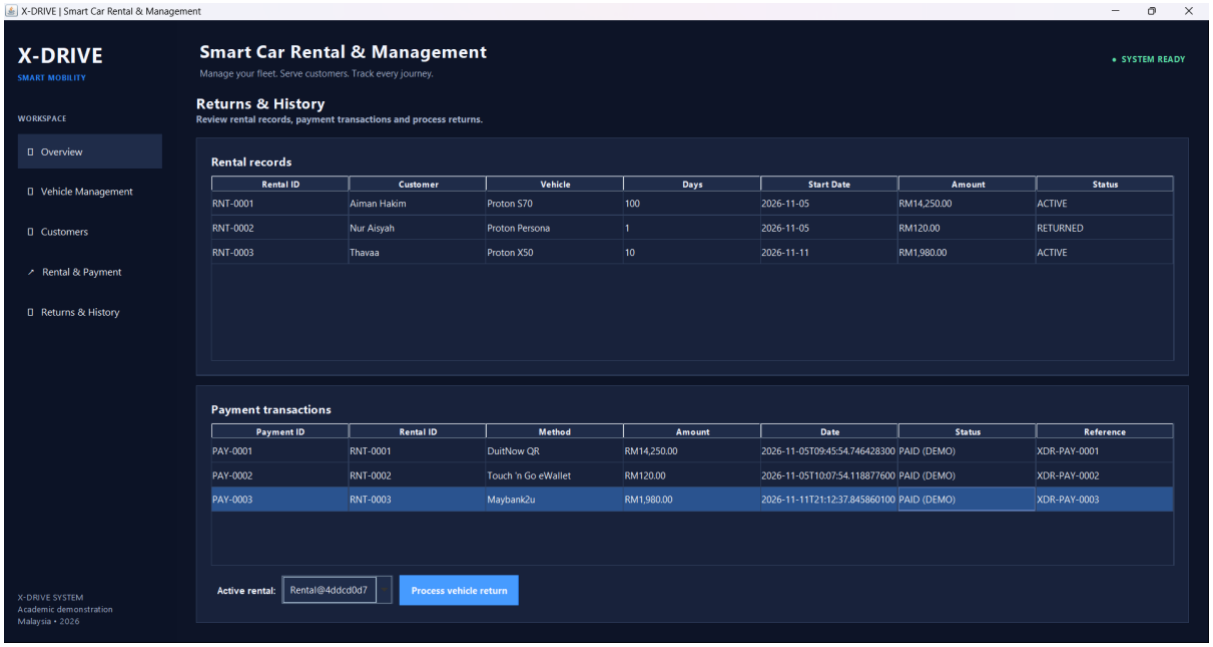


Figure 6.5.1: Returns and Rental History

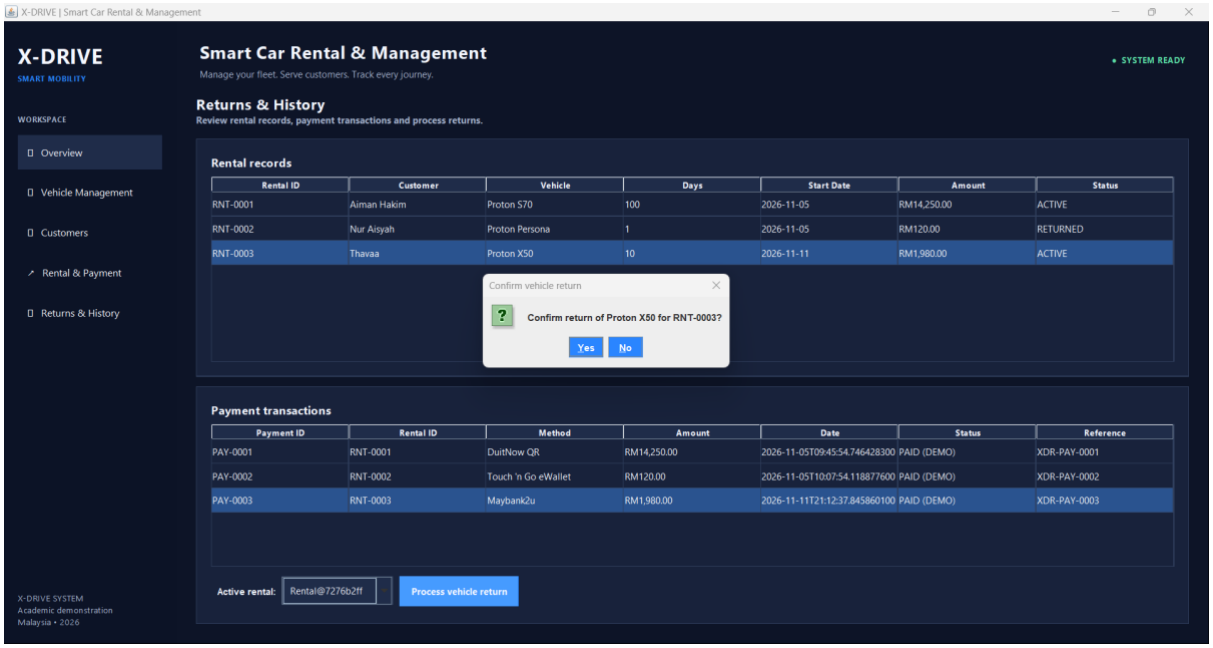


Figure 6.5.2: Returns and Rental History

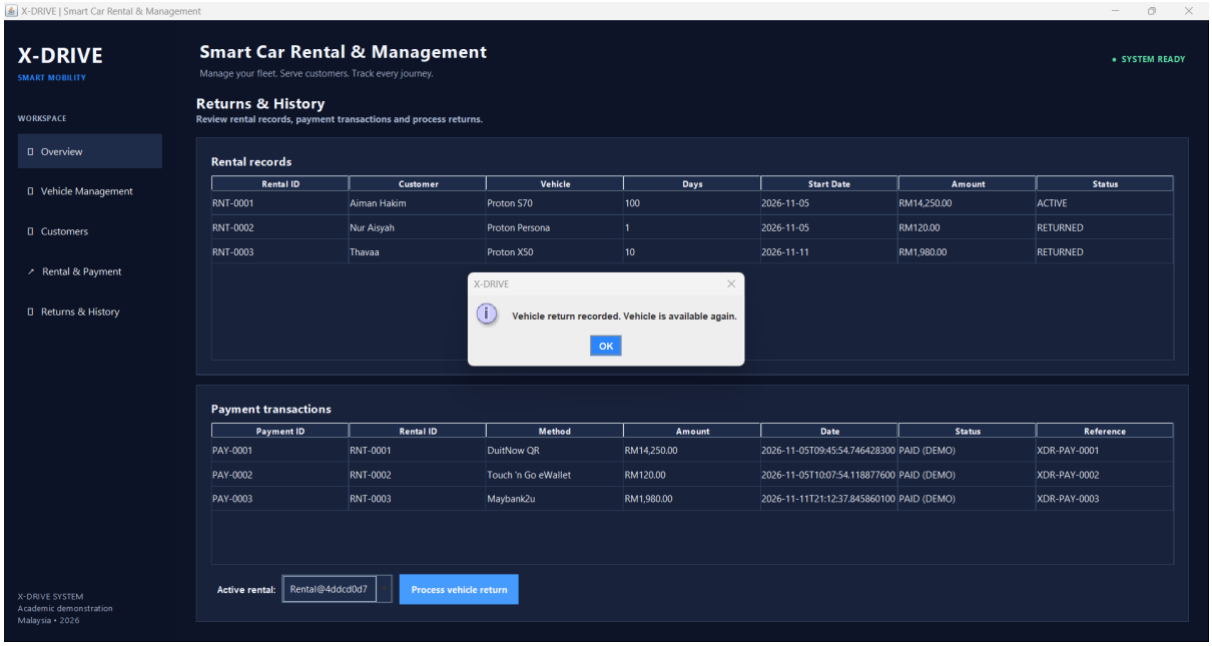


Figure 6.5.3: Returns and Rental History

The returns section allows users to process the return of rented vehicles. Once a rental is returned, the vehicle availability status can be updated.

The history section allows users to review previous rental and payment records.

These features help maintain an organised record of rental activities.

7.0 CONTROL STRUCTURES

Control structures are used to determine how the program executes instructions and responds to different conditions.

7.1 If-Else Statements

The system uses conditional statements to check different conditions during its operations.

Examples include checking vehicle availability before allowing a rental, validating input information, and checking rental status before processing a return.

Conditional statements help prevent invalid operations and allow the application to respond appropriately to different situations.

7.2 Loops

Loops are used to repeat instructions when processing collections of information.

For example, loops can be used to display vehicle records, customer records, rental transactions, and payment information.

This reduces the need to write repeated statements for every individual record.

7.3 Event Handling

The graphical interface uses event handling to respond to user actions.

For example, when a user clicks a button, the application executes the corresponding operation. This allows users to interact with the system through the graphical interface.

8.0 TESTING AND RESULTS

Testing was carried out to check whether the main functions of X-DRIVE operate as expected.

The application was compiled and executed using the Java Development Kit. The graphical interface opened successfully, and the main system operations were tested.

8.1 Functional Testing

No.	Test Description	Expected Result	Result
1	Launch the application	Main application window opens	Passed
2	View vehicle records	Vehicle information is displayed	Passed
3	Add a new vehicle	New vehicle appears in the vehicle list	Passed
4	Register a customer	Customer information is added	Passed
5	Create a rental	Rental transaction is created	Passed
6	Record payment	Payment information is processed by the application	Passed
7	Return a rented vehicle	Rental return is processed and vehicle availability is updated	Passed
8	View rental history	Rental records are displayed	Passed
9	Navigate between sections	Relevant system pages are displayed	Passed

8.2 Sample Rental Calculation

A sample rental transaction was tested using the following details:

Item	Value
Vehicle	Proton X50
Vehicle Type	SUV
Daily Rental Rate	RM180
Rental Duration	10 Days
Basic Rental Amount	RM1800
Surcharge	10%
Surcharge Amount	RM18
Total Amount	RM1980

Calculation:

Basic Rental Amount:

RM180 × 10 = RM1800

Surcharge:

RM180 × 10% = RM18

Total Rental Amount:

RM1800 + (RM18 ×10) = RM1980

The application successfully displayed the calculated total for this sample transaction.

8.3 Testing Summary

Based on the functional testing carried out, the main application sections operated as expected. The system successfully launched, displayed vehicle records, supported customer registration, processed rental and payment operations, and handled vehicle returns and rental history.

The testing demonstrated that the main functions of the X-DRIVE application were working during the project demonstration.

9.0 CONCLUSION

In conclusion, X-DRIVE successfully demonstrates the application of Object-Oriented Programming concepts in developing a Smart Car Rental and Management System using Java. The project applies encapsulation, inheritance, polymorphism, abstraction, constructors, getter methods, setter methods, and object creation through a structured class-based design.

The system also provides practical features such as vehicle management, customer registration, rental processing, payment recording, vehicle returns, and rental history. The graphical user interface developed using Java Swing makes the application easier to navigate and operate.

Through this assignment, the group gained a better understanding of how different classes and objects interact in a real-world application. The project also provided practical experience in designing, implementing, compiling, testing, and improving a Java application.

In the future, X-DRIVE could be improved by introducing database connectivity, user authentication, automated receipt generation, online payment gateway integration, and additional reporting features. Overall, the project demonstrates how Object-Oriented Programming can be used to develop an organised and functional car rental management application.

APPENDIX :

GitHub Repository/Link:

https://github.com/nithannn/-X-DRIVE-Smart-Car-Rental-System.git

The repository contains the Java source files and project documentation required to understand, compile, and run the application. The project was developed using Java and Java Swing.

To run the application, download the repository, open the project folder in a Java development environment, compile the Java source files, and execute the main application class, CarRentalApp.

Compilation Command:

```
javac *.java
```

Execution Command:

```
java CarRentalApp
```

REFERENCES

1. Oracle. (n.d.). *Java Documentation*. https://docs.oracle.com/en/java/
2. Oracle. (n.d.). *The Java Tutorials – Object-Oriented Programming Concepts*. https://docs.oracle.com/javase/tutorial/java/concepts/
3. Oracle. (n.d.). *The Java Tutorials – Creating a GUI With Swing*. https://docs.oracle.com/javase/tutorial/uiswing/
4. Oracle. (n.d.). *Java Platform, Standard Edition Documentation*. https://docs.oracle.com/en/java/javase/19/
5. W3Schools. https://www.w3schools.com/java/default.asp